COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/7/26	**DATE SUBMITTED:**	10/7/26
NAME:	Reyes, Cyrill Joferson B. Silvoza, Keith Nicole B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 **Basic Algoritm Analysis**

1. PROCEDURE OUTPUT

Reyes

ILO A

Analysis: The graph demonstrates quadratic growth $O(n^2)$ following $f(n) = n^2 + n$, where doubling the array size nearly quadruples the total operations. The accelerating operation count—increasing by 310 between $n = 10$ and 20, then jumping by 510 between $n = 20$ and 30—shows that the algorithm rapidly loses efficiency as dataset sizes scale.

Total number of comparisons in the worst case: $m \cdot n$ (or n^2 when $m = n$).

Silvoza

THE GRAPH EXHIBITS QUADRATIC GROWTH DEFINED BY N SQUARED PLUS N. AS ARRAY SIZE TRIPLES
FROM 10 TO 30, OPERATIONS INCREASE SHARPLY FROM 110 TO 930. THE ACCELERATING GROWTH
RATE HIGHLIGHTS POOR SCALABILITY FOR LARGE DATASETS.

ILO B

HOA11p1 > main.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
// constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

// getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

// two people are same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }
// the ordering of a set of people is defined by their name
    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

// the IDs of student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

// generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}
```

```cpp

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

// return true if the student is found in the sequence and
// he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        else if (mid_element < P)
            std::advance(first, mid_element_index);

// student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}

void search_test(int size, Student p) {
    std::vector<Student> people;

// create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());
// to measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

// stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

```

```
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") does not need vaccination." << std::endl;
}

int main() {
    // generate a student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

```
PS C:\Users\TIPQC\OneDrive\Documents\CPE12S4 - Reyes>
exe' '--stdin=Microsoft-MIEngine-In-t5hd0vgn.b4c' '--st
bgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Time taken to search = 1 microseconds
Student (702 801) needs vaccination.
Time taken to search = 2 microseconds
Student (702 801) needs vaccination.
Time taken to search = 1 microseconds
Student (702 801) needs vaccination.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S4 - SILVOZA>
s-vscode.cpptools-1.34.4-win32-x64\debugAdapters\bin\Win
' '--stdout=Microsoft-MIEngine-Out-25sibaen.efp' '--stde
e-Pid-aafv0hgr.g3r' '--dbgExe=C:\msys64\ucrt64\bin\gdb.e
Time taken to search = 1 microseconds
Student (599 142) needs vaccination.
Time taken to search = 1 microseconds
Student (599 142) needs vaccination.
Time taken to search = 2 microseconds
Student (599 142) needs vaccination.
PS C:\Users\TIPQC\OneDrive\Documents\CPE21S4 - SILVOZA>
```

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	1 microsecond	Time taken to search = 1 microseconds Student (702 801) needs vaccination.	Search finishes in 1 miliseconds student (702, 801) is recognized as

			needing vaccination.
10,000	2 microseconds	Time taken to search = 2 microseconds Student (702 801) needs vaccination.	Search completes in 2 miliseconds; increasing input size tenfold causes virtually no change in execution speed.
100,000	1 microsecond	Time taken to search = 1 microseconds Student (702 801) needs vaccination.	Search completes in 1 miliseconds; runtime remains flat despite a $100 \times$ increase in dataset size, confirming $O(\log N)$ scalability.

Silvoza

INPUT SIZE (N)	EXECUTION SPEED	SCREENSHOT	OBSERVATIONS
1,000	1 microseconds	Time taken to search = 1 microseconds Student (599 142) needs vaccination.	EXECUTES INSTANTLY IN ZERO MICROSECONDS DUE TO LOGARITHMIC EFFICIENCY REQUIRING AROUND TEN COMPARISONS
10,000	1 microseconds	Time taken to search = 1 microseconds Student (599 142) needs vaccination.	INCREASES TO ONLY ONE MICROSECOND DESPITE A TEN TIMES DATASET GROWTH REQUIRING AROUND THIRTEEN COMPARISONS
100,000	2 microseconds	Time taken to search = 2 microseconds Student (599 142) needs vaccination.	TAKES JUST TWO MICROSECONDS REQUIRING AROUND SEVENTEEN COMPARISONS DEMONSTRATING CLASSIC LOGARITHMIC BINARY SEARCH SCALABILITY.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

Theoretical Analysis

Time Complexity: The algorithm uses nested loops to compare all element pairs. In the worst case (when all elements are unique), the inner loop executes $\frac{n(n-1)}{2} = \frac{n^2-n}{2}$ times, giving a worst-case time complexity of $O(n^2)$. The best-case complexity is $O(1)$ if a duplicate is found on the first comparison.
Space Complexity: Auxiliary space complexity is $O(1)$ because element comparisons occur in-place without additional memory allocations.

Experimental Analysis

- $N = 1,000$: $499,500$ comparisons | Execution Time: 206 milliseconds
- $N = 5,000$: $12,497,500$ comparisons | Execution Time: 4,805 milliseconds
- $N = 10,000$: $49,995,000$ comparisons | Execution Time: 19,504 milliseconds
- $N = 20,000$: $199,990,000$ comparisons | Execution Time: 81,090 milliseconds

Analysis and Comparison

- The experimental runtimes align directly with the theoretical $O(n^2)$ model. Scaling input size N tenfold (from 1,000 to 10,000) results in a $100\ \times$ increase in total comparisons ($499.5k \rightarrow 49.995M$), which produced a corresponding $\approx 94.6\ \times$ runtime increase in empirical testing.

Problem 2
Theoretical Analysis:
power(x, n): Decreases the exponent linearly $(n - 1)$ with each recursive call. The recurrence relation $T(n) = T(n - 1) + O(1)$ results in a linear time complexity of $O(n)$ and a stack space complexity of $O(n)$.
The function
brpower(x, n): Halves the exponent (n/2) per recursive call. Recurrence relation $T(n) = T(n/2) + O(1)$ yields a logarithmic time complexity of $O(\log n)$ and a stack space complexity of $O(\log n)$.

Experimental Analysis

- $N = 1,000$: rpower = $2,368ns$ | brpower = $148ns$
- $N = 10,000$: rpower = $169,334ns$ | brpower = $247ns$
- $N = 100,000$: rpower = *Stack Overflow* | brpower = $151ns$
- $N = 1,000,000$: rpower = *Stack Overflow* | brpower = $217ns$

Analysis and Comparison:
Experimental timing shows the significant efficiency benefit of binary exponentiation $O(\log n)$ compared to linear recursion $O(n)$. Though rpower increases linearly and causes a stack overflow at $N \geq 100,000$ because of deep call frames, brpower handles even $N = 1,000,000$ in less than $250ns$ utilizing fewer than 20 total recursive frames.

Silvoza

PROBLEM 1

- **THEORETICAL ANALYSIS**

Basic Operation: Element comparison (A[i] == A[j]).
Operation Count: $T(n) = (n - 1) + (n - 2) + ... + 1 = n(n - 1)/2 = 0.5n^2 - 0.5n$.
Worst-Case Complexity: $O(n^2)$ when all elements are unique.
Best-Case Complexity: O(1) when a duplicate is found at A[0] and A[1].
Auxiliary Space Complexity: O(1).

- **EXPERIMENTAL ANALYSIS**

n = 1,000 | Theoretical: 499,500 | Time: 0.024 s | Growth Ratio: —
n = 2,000 | Theoretical: 1,999,000 | Time: 0.095 s | Growth Ratio: ~3.96x
n = 4,000 | Theoretical: 7,998,000 | Time: 0.385 s | Growth Ratio: ~4.05x
n = 8,000 | Theoretical: 31,996,000 | Time: 1.540 s | Growth Ratio: ~4.00x

- **ANALYSIS AND COMPARISON**

Doubling input size n causes execution time to increase by roughly 4x (2^2), directly confirming the theoretical $O(n^2)$ quadratic time complexity.

PROBLEM 2

- **THEORETICAL ANALYSIS**

Algorithm rpower (Linear Recursion):

- Recurrence: $T(n) = T(n - 1) + O(1)$
- Time Complexity: O(n)
- Space Complexity: O(n) call stack space

Algorithm brpower (Divide and Conquer):

- Recurrence: $T(n) = T(n / 2) + O(1)$
- Time Complexity: O(log n)
- Space Complexity: O(log n) call stack space

- **EXPERIMENTAL ANALYSIS**

n = 1,000 | rpower: 0.000450 s | brpower: 0.000015 s | Speedup: ~30x

n = 10,000 | rpower: 0.005100 s | brpower: 0.000028 s | Speedup: ~182x

n = 50,000 | rpower: 0.028500 s | brpower: 0.000038 s | Speedup: ~750x

n = 100,000 | rpower: 0.062000 s | brpower: 0.000045 s | Speedup: ~1377x

- **ANALYSIS AND COMPARISON**

rpower scales linearly O(n) and risks recursion stack overflow for large n. brpower scales logarithmically O(log n), executing drastically faster with minimal memory usage.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, formatting, text summarization, and explanation of algorithm analysis concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to refine written explanations, format lab result tables, and assist in debugging C++ syntax errors. All experimental data execution, measurement results, theoretical evaluations, and final analytical conclusions were independently performed and verified, ensuring the AI served strictly as a supportive tool without replacing original thinking or authorship.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.